COSC204
Computer Systems

6809 Hello World

Overview

- Last Lecture
  - 6809 assembly syntax
- This Lecture
  - Assembly language programming
  - Source
    - Any book on 6809
      - Lance A. Leventhal (1981) 6809 Assembly Language Programming, Osborne McGraw-Hill, ISBN: 9780079310354
      - Card D. Warren (1980) The MC6809 Cookbook, Tab Books
    - 6809 datasheet
- Next Lecture
  - Calling conventions

Assembly Language

- Programming in assembly language is tricky
  - You can't declare variables, you can only use registers
    - You have a tiny number of pre-defined registers
    - Which are *like* variables, but only hold numbers
  - The only loop construct is like a GOTO
  - You can only compare integers or bytes
  - You can only branch if a comparison matches certain conditions
    - The last comparison was equal, not equal, greater, less, etc.
  - Routines only kinda-sorta exist
    - You have to "roll your own" from the parts the CPU provides
  - All registers (variables) are global

Instruction Syntax

- Normally case insensitive
- Syntax

  <label:> <opcode> <operands> <;comment>

- Labels
  - Start with a letter and ends with a colon (can be alphanumeric)
  - Some assembles don't require the colon
- Operands

  # immediate (if no # is given then it's an address)

  $ hex value

  % binary value

Hello World

- We’re going to look at Hello World because it demonstrates several essential principles
  - Routines (functions, methods, subroutines)
    - Declaration and calling conventions
  - Iteration
    - Branch instructions

Routines

- We declare a routine like this

```
ROUTINE:
    RTS
```

- Routines do not take parameters
- You cannot declare local variables
- All registers are global
- `RTS` means return from subroutine
  - So this routine does nothing

Routines

- There are three ways to call a routine, `LBSR`, `BSR`, `JSR`. The differences are subtle. We'll use `BSR`

```
BSR ROUTINE
```

- There are no parameters to routines
  - The registers are global

Iteration

- All iteration is with GOTOs (branch instructions)
  - for (a = 0; a < 6; a++)

```
    CLRA          ; A = 0
MORE:
    CMPA #$06     ; compare A to value $06
    BEQ DONE      ; if equal,branch to done
    INCA          ; A = A + 1
    BRA MORE      ; branch to more
DONE:
```

Iteration

- A comparison only compares then sets flags in `CC`
- You then (optionally) branch on the result
  - `BEQ`, `BGE`, etc.
- But other instructions set flags too
  - `LDA`, etc
- So you can write

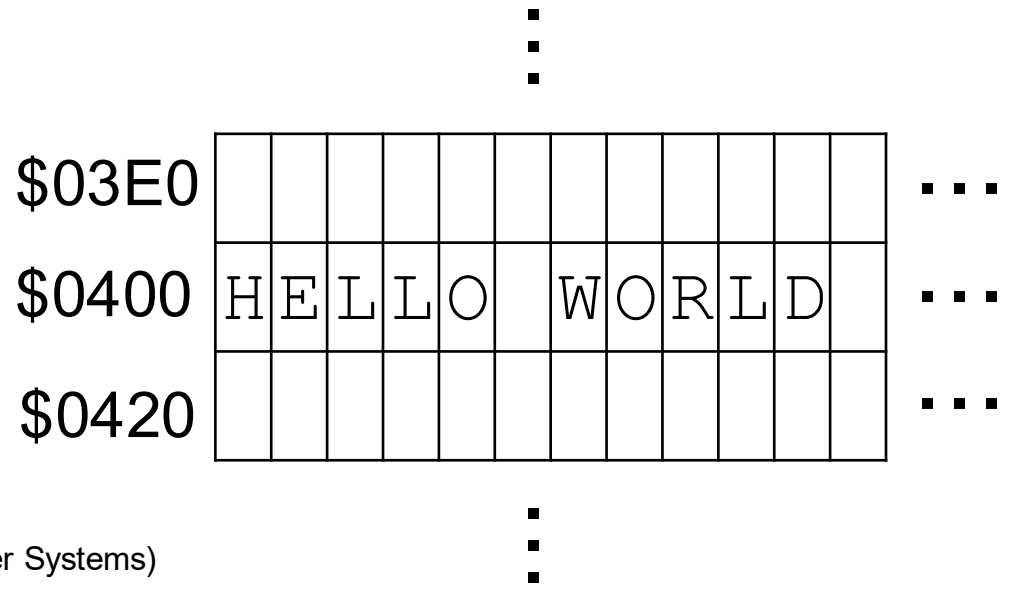
```
LDA $45
BEQ SOMEWHERE
```

Hello World

- We're going to need a routine that, given a string, will print each character one at a time
  - We know how to write routines and iterate
- We need to know how to print to the screen
  - We're going to use the 6809.uk simulator

6809.uk text screen

- The screen on the 6809.uk simulator:
  - 16 lines by 32 columns and each location is 1 byte
  - So the screen is 512 bytes in size
  - Starts at location $0400
- Is not ASCII, but UPPERCASE is mostly correct



6809.uk text screen

- We can write an ‘H’ to the top left of the screen

```
LDA #'H'
STA $0400
```

- And an ‘E’ beside that

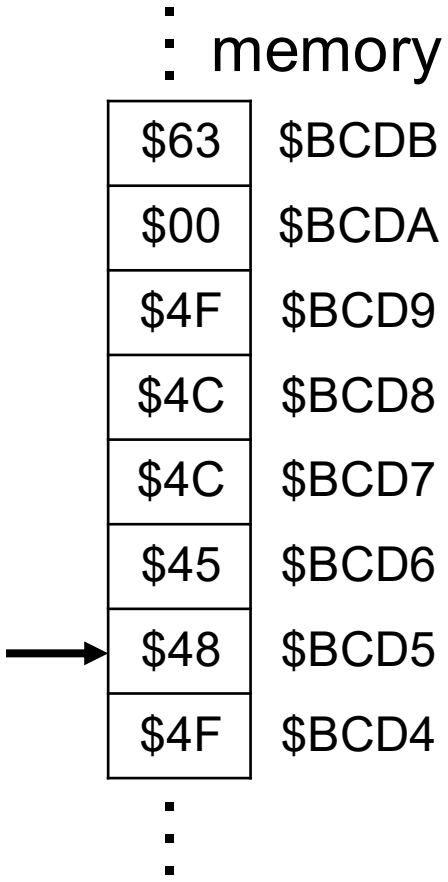
```
LDA #'E'
STA $0401
```

- And so on

- But how about a routine that prints a string to the top left of the screen?

Print Routine

- We'll pass the address of the string in `Y`
  - Recall, that the memory is just an array of bytes
  - And so the string we want to print must be somewhere in that array
  - By "address of the string" we mean:
    - What is the index into the memory array that holds the first character of the string
    - We already know this, because we put it there
      - We'll see how to do that soon
- We'll use a $00 to end the string
  - Just like in C.



Algorithm

- Print a String (`PUTS`)
  - Strings are terminated with a $00 character (C or ASCIIZ strings)

- Algorithm (address of string passed in `Y`)

  Start:

  Load a character from memory[Y]

  Move on to the next character (Y++)

  Compare the character to $00

  If equal exit this routine (branch to the end of this routine)

  Write the character to the screen

  Move on to the next screen location

  GOTO start

PUTS()

```
;
; Routine: PUTS
; Pass the address of the string in Y
;
PUTS:
   LDX #$0400  ; start of screen
MORE:
   LDA 0,Y      ; current char -> A
   LEAY 1,Y     ; increment Y
   CMPA #$00    ; was it a zero?
   BEQ DONE     ; if it was 0 then return
   STA 0,X      ; write
   LEAX 1,X     ; increment X
   BRA MORE     ; repeat
DONE:
   RTS        ; return from this routine
```

Defining Constants

- But there's better (clearer, smaller, faster) ways of doing some of that

```
LDX #$0400  ; start of screen
```

- That's a hard coded constant (magic number) which we already know is bad, so let's define a constant

```
SCREEN EQU $0400 ; start of screen
LDX #SCREEN
```

- The syntax for defining a constant is

  <constant> EQU <value>

Post Increment

- We loaded `A` with 0 bytes past where `Y` points then incremented `Y` (using `LEAY`).

```
LDA 0,Y         ; current char -> A
LEAY 1,Y        ; increment Y
```

- But we can do that in one instruction

```
LDA 0,Y+
```

- And we did the same with `X`

```
STA 0,X         ; write
LEAX 1,X        ; increment X
```

- Which becomes

```
STA 0,X+
```

PUTS()

```
;
; Routine: PUTS
; Pass the address of the string in Y
;
SCREEN EQU $0400 ; start of screen

PUTS:
   LDX #SCREEN   ; start of screen
MORE:
   LDA 0,Y+      ; current char -> A
   CMPA #$00     ; was it a zero?
   BEQ DONE      ; if it was 0 then return
   STA 0,X+      ; write
   BRA MORE      ; repeat
DONE:
   RTS           ; return from this routine
```

CC Set on Load

- We have

```
LDA 0,Y+    ; current char -> A
CMPA #$00   ; was it a zero?
BEQ DONE    ; if it was 0 then return
```

- But `CC` is set on an `LDA`, so if we loaded a zero then the `Z` flag will be set so we don't need the `CMPA`

```
LDA 0,Y+    ; current char -> A
BEQ DONE    ; if it was 0 then return
```

- Now it's looking a lot cleaner

Registers Are Global

- We need to think about what happens to the registers. We have changed `X`, `Y`, and `A`
  - So we need to add an instruction to save them at the start

    ```
    PSHS A,X,Y
    ```

  - And to restore them at the end

    ```
    PULS A,X,Y
    ```

- When we look at “calling conventions” we’ll see that there are other ways of doing this.

PUTS()

```
;
; Routine: PUTS
; Pass the address of the string in Y
;
SCREEN EQU $0400 ; start of screen

PUTS:
   PSHS A,X,Y    ; save A, X, and Y
   LDX #SCREEN   ; start of screen
MORE:
   LDA 0,Y+      ; current char -> A
   CMPA #$00     ; was it a zero?
   BEQ DONE      ; if it was 0 then return
   STA 0,X+      ; write
   BRA MORE      ; repeat
DONE:
   PULS A,X,Y    ; restore A, X, Y
   RTS           ; return from this routine
```

Calling our Routine

- First we declare the string we want to print

```
MESSAGE:
    .BYTE "HELLO WORLD", $00
```

- We've used a label (`MESSAGE`) to mark a sequence of bytes stored in memory
- We put the 0 on the end because our puts routine only stops when it sees a `$00`

Calling our Routine

- We use the label to get the location (in our array called memory) where the byte sequence is stored

```
LDY #MESSAGE
```

- Then we call our routine

```
BSR PUTS
```

The Program

- We'll mark the start of the program with a label called START

```
START:
    LDY #MESSAGE
    BSR PUTS
    RTS

MESSAGE:
    .BYTE "HELLO WORLD", 0
```

- That's it!

The Assembler

- The program that “compiles” assembly language programs is called an ***assembler***
- It translates the assembly instructions directly into machine code
- There is a direct correspondence between mnemonic assembly language and the machine code bytes (one mnemonic becomes one sequence of machine code bytes)
- You can (mostly) go the other way too
  - A ***disassembler*** translates the machine code back into assembly language (but it can’t guess label names)

Assembly Language Listing

- Assemblers can produce a “listing” file that gives the location, machine code, and assembly of the program

Location	Label	Machine Code	Assembly Language
4000:	PUTS	34 32	PSHS A,X,Y
4002:		8E 04 00	LDX #$0400
4005:	MORE	A6 A0	LDA ,Y+
4007:		81 00	CMPA #$00
4009:		27 04	BEQ DONE
400B:		A7 80	STA ,X+
400D:		20 F6	BRA MORE
400F:	DONE	35 32	PULS A,X,Y
4011:		39	RTS

Homework

- What is the difference between `BEQ` and `LBEQ`?
  - Why do we prefer `BEQ`?
- Why do we prefer `BSR` to `JSR`?
- How is our `PUTS` routine different from a string copy routine such as `strcpy()`?
- Change our `PUTS` routine into a routine called `CLS` that clears the 512 byte screen starting at $0400